

Laboration 5

Objektorienterad programmering

För att introducera objektorienterad programmering så tittar vi på ett beräkningsproblem som dyker upp i modern molekylärbiologi. När man analyserar DNA-sekvenser så vill man ibland veta hur pass två sekvenser överlappar (förenklat: hur pass mycket DNA de har gemensamt). Du ska skriva enkel objektorienterad kod för att stödja sådan analys.

5.1 Lärandemål

- Du ska kunna skriva en egen klass, och metoder som hör till klassen, för att lösa programmeringsuppgifter.
- Du ska kunna använda använda funktioner som argument till funktioner.
- Du ska kunna använda särfall (eng: *exceptions*) för att hantera fel.

5.2 Förkunskaper

För att lösa uppgiften bör du ha läst på om objektorientering och högre ordningens funktioner.

5.3 Ej accepterade tekniker

Du kan använda alla tekniker vi har diskuterat i kursen.

5.4 Given kod

Det finns en kodstubbe `dna.py` på kurswebben som du ska använda som startpunkt. Där hittar du en gnutta kod som ska hjälpa dig att komma igång, samt lite kod för att hjälpa dig testa om du löst uppgiften eller inte. Testkoden har två syften.

- (1) Det ska bli lättare och snabbare för dig att avgöra om du gjort rätt. Det är ofta praktiskt att ha testkod av detta slag. I kursen kommer vi att introducera "riktig", systematisk, testning.
- (2) Testerna är en kodad specifikation av hur din kod ska fungera. Om anropet `test_all()` skriver ut `Yay, all good` så är du antagligen klar, men kom ihåg att det alltid är bra att testa koden själv!

5.5 Uppgifter

5.5.1 Uppgift 1: Definiera en klass för DNA-sekvenser

Definiera klassen `DnaSeq` som har två attribut: en unik identifierare (även känd som ett *accession*, på engelska) och en DNA-sekvens. Kalla attributen `accession` och `seq`. Det ska vara möjligt att skapa ett `DnaSeq`-objekt så här:

```
my_seq = DnaSeq('abc123', 'ACGTACGT')
```

Krav

- Din klass ska ha en konstruktor (`__init__`, som givet i startkoden) och två metoder: `__len__` och `__str__`. Metoder med dessa namn har särskild mening i Python: det går att tillämpa funktionerna `len` och `str` på ett objekt från en klass med dessa definierade.
- I denna övning ska `__len__` returnera längden på DNA-sekvensen (dvs, strunta helt i `accession`), och `__str__` ska returnera en "etikett" för praktiska utskrifter på formen `<DnaSeq accession='abc123'>` om tillämpat på exempelobjektet ovan.
- Instansiering med en tom identifierare (`accession`) eller tom DNA-sekvens ska orsaka särfallet `ValueError`. Dvs, anrop som `DnaSeq(' ', None)` ska resultera i ett särfall.
- Metoderna ska inte skriva ut något, bara returnera värden.
- När du är klar ska anropet `test_class_DnaSeq()` generera utskriften `DnaSeq passed` och inga särfall eller fel ska märkas.

5.5.2 Uppgift 2: Läs DNA-sekvenser från fil

Implementera funktionen `read_dna()`, som givet ett filnamn som parameter returnerar en lista med `DnaSeq`-objekt som representerar sekvenserna i filen. Filformatet är som följer:

- En fil är en rad med "poster" (eng: *records*).
- Varje post innehåller en id-rad (eng: *accession line*) och en sekvensrad.
- En identifierare är unik i filen, representeras av en enkel sträng, och kommer efter ett större-än-tecken (>) som inte är del av identifieraren.
- En sekvens är en sträng av bokstäverna A, C, G, och T.
- Det kan finnas tomrader mellan posterna. Dessa tomrader ska ignoreras. Tips: tomrader har inte längd 0, eftersom det finns alltid ett nyradstecken (skrivs som `\n`) när man läser in en tomrad.

Här är ett enkelt exempel:

```
>s123
ACGGACGT
>abc
GATTACA

>X20
AAAAAAGAATTACCCACACACAC
```

Kodmappen på kurswebben innehåller filer som du kan testa din kod med. Filerna har suffixet ".fa" och är vanliga textfiler som du kan öppna med en programmeringseditor som Spyder och enkla textediteringsprogram som Notepad och TextEdit. Filerna ex1.f, ex2.f, och sars_cov_2.f används av testkoden, så du måste ladda ner dem.

5.5.3 Uppgift 3: En funktion för att detektera överlapp

Vi vill kunna identifiera *överlapp* mellan två DNA-sekvenser med en funktion kallad `check_exact_overlap`. Vi säger att sekvensen a överlappar med sekvensen b om det finns ett suffix till a av någon längd L som är identisk med ett prefix i b av längd L . Definitionen är inte kommutativ: a kan överlappa med b utan att b överlappar med a .

Exempel 1: Här nedan visar vi alla sätt som DNA-sekvensen AAACCC kan överlappa med CCCCGATT.

AAACCC	AAACCC	AAA
CCC	CC	CCC
CGATT	CGATT	CGATT
överlappslängd 1	överlappslängd 2	överlappslängd 3

Sekvenserna har inget överlapp längre än 3. Om vi testar överlappslängd 4 och 5, så är det inte ett identiskt överlapp:

AA	AA
CCC	CCC
CGATT	CGATT
Inget överlapp	Inget överlapp

Exempel 2: Jämför sekvenserna från exempel 1 i motsatt ordning. Sekvensen "CCCGGATT" har inget överlapp med "AAACCC", eftersom det inte finns något suffix för "CCCGGATT" som är identiskt med ett prefix för "AAACCC".

CCCGGATT	CCCGGATT	CCCGGATT	
AAA	AA	AAA	
CGATT	CGATT	CGATT	
Inget överlapp	Inget överlapp	Inget överlapp	...

Exempel 3: GAGATCAT och ATCATTT har två överlapp, ett med längd 2 och ett med längd 5.

GAGATCAT	GAGATCAT
ATCATTT	ATCATTT
överlappslängd 2	överlappslängd 5

Exempel 4: AAACCC överlappar med ACCC. Hela ACCC överlappar faktiskt med längd 4, och det är det enda överlappet.

AAACCC	AAACCC	AAA	AA
ACCC	ACCC	ACCC	ACCC
Inget överlapp	Inget överlapp	Inget överlapp	length 4 overlap

Implementera `check_exact_overlap` med tre parametrar: två `DnaSeq`-objekt (motsvarande *a* och *b* ovan) och en minimilängd. Överlapp kortare än minimilängden ska ignoreras.

Krav

- Parametern för minsta överlapp ska ha skönsvärdet (eng: *default value*) 10.
- Längden på längsta överlapp ska returneras, men längden får inte vara kortare än minimilängd.
- Returnera 0 om inget överlapp hittas.
- Funktionen ska ha en dokumentationssträng.

5.5.4 Uppgift 4: Högre ordningens funktion för att detektera överlapp

Implementera funktionen `overlaps` med två parametrar:

- En lista med `DnaSeq`-objekt.
- En funktion för att detektera överlapp.

Ett exempel på överlappsfunktion är förstås `check_exact_overlap`, men vilken funktion som helst som tar samma parametrar ska kunna användas. I testkoden så konstrueras alternativa överlappsfunktioner. Om `lst` är en lista med `DnaSeq`-objekt så ska `overlaps(lst, check_exact_overlap)` vara ett giltigt funktionsanrop.

Alla detekterbara överlapp mellan par av sekvenser i indatalistan ska returneras.

Krav

- Returnera en uppslagstabell (`dict`) med uppslagstabeller (så en “dict of dicts”) som innehåller längder på överlapp. Om ett anrop till `overlaps` ger `d` som resultat, och sekvenserna med id `s1` och `s2` överlappar med längd 10, då ska `d['s1']['s2'] == 10` vara sant. Om dessa två sekvenser var enda indata så skulle den nästlade uppslagstabellen kunna se ut så här: `{ 's1': { 's2': 10 } }`.
- Det ska inte finnas några tomma element i den returnerade uppslagstabellen. Om `s1` inte överlappar med någon annan sekvens så ska man inte hitta `s1` i returdata.
- Inget ska skrivas ut av funktionen.
- Anropet `test_overlap()` ska ge utskriften `overlap code passed` och inga fel ska uppstå när din kod är klar.

5.6 För den nyfikne

- Filformatet vi använder är en förenklad version av formatet “FASTA” som används i stor utsträckning i bioinformatiken. Därav suffixet “.fa” på testdata.
- Man kan säga att DNA har riktning, men i praktiken är denna riktning oftast inte känd. Man måste då överväga både fram- och bakriktning på en DNA-sekvens, och dessutom med så kallat “omvänt komplement” av DNA (eng: *reverse complement DNA*).
- Beräkning av överlapp är kärnan i sammansättning av arvsmassa (eng: *genome assembly*), men man kan då inte titta på perfekta överlapp (som vi gjort här) utan måste acceptera att det finns fel och små skillnader i DNA-läsningarna. Eftersom datamängden i sammansättning av arvsmassa tenderar att vara mycket stor så krävs snabba och smarta algoritmer särskilt anpassade för ändamålet.

5.7 Inlämning

Lämna som vanligt in koden i rätt inlämningsmodul på kurshemsidan. Det räcker att lämna in en fil med den slutgiltiga versionen av koden. Kommentera gärna koden där du har gjort ändringar.

Muntlig redovisning: Den kod du har lämnat in *kan* behöva redovisas inför lärare vid nästa labbtillfälle.